

NodeAlert IoT: Sistema de Monitoreo Ambiental para la Detección Temprana de Condiciones Peligrosas

Un proyecto académico presentado en la Universidad Nacional de Rafaela

Eloy Kirov, Francisco Alemandi, Palavecino Sebastian

Universidad Nacional de Rafaela (UNRaf)
Ingeniería en Computación

Docentes: Ing. Milton Pozzo, Ing. Matias Wanzienried

26 de noviembre de 2025

1. RESUMEN

El presente trabajo expone la evolución del sistema *FireSafe* hacia una arquitectura de Internet de las Cosas (IoT) distribuida, diseñada para el monitoreo ambiental crítico y la detección temprana de riesgos. A diferencia de su predecesor, esta versión implementa una topología de red en la que múltiples nodos periféricos basados en el microcontrolador ESP32 realizan el procesamiento en el borde (*Edge Computing*), gestionando la concurrencia de tareas mediante el sistema operativo de tiempo real FreeRTOS (?).

La infraestructura se apoya en un protocolo de mensajería asíncrona MQTT bajo un modelo de publicación/suscripción, garantizando una comunicación eficiente y escalable entre los nodos y el servidor central en una Raspberry Pi. El sistema integra un *backend* desarrollado en Django para la persistencia de datos y un *dashboard* interactivo en React que permite la visualización en tiempo real y la gestión de alertas. Los resultados demuestran que el análisis de circuitos propuesto por ? y la implementación de interfaces de tiempo real (?) permiten una respuesta inmediata ante fluctuaciones térmicas o presencia de gases combustibles, validando la viabilidad de soluciones industriales de bajo costo.

Palabras clave: IoT distribuido, MQTT, FreeRTOS, monitoreo ambiental, sistemas de tiempo real.

2. ABSTRACT

This paper presents the evolution of the *FireSafe* system towards a distributed

Internet of Things (IoT) architecture, designed for critical environmental monitoring and early hazard detection. Unlike its predecessor, this version implements a network topology where multiple ESP32-based peripheral nodes perform edge computing, managing task concurrency through the FreeRTOS real-time operating system (?).

The infrastructure is supported by the MQTT asynchronous messaging protocol under a publish/subscribe model, ensuring efficient and scalable communication between nodes and the central server on a Raspberry Pi. The system integrates a Django-developed backend for data persistence and a React-based interactive dashboard for real-time visualization and automatic alert management. The results demonstrate that the circuit analysis proposed by ? and the implementation of real-time interfacing (?) allow for an immediate response to thermal fluctuations or the presence of combustible gases, validating the feasibility of low-cost industrial solutions.

Keywords: Distributed IoT, MQTT, FreeRTOS, environmental monitoring, real-time systems.

3. INTRODUCCIÓN

La seguridad en entornos habitacionales e industriales representa uno de los pilares fundamentales del bienestar humano y la preservación de activos materiales. En particular, la prevención y detección temprana de incendios y fugas de gases peligrosos constituye un desafío técnico crítico, dado que el tiempo de respuesta es inversamente proporcional a la magnitud de los daños resultantes. Históricamente, la ingeniería ha abordado esta problemática mediante dispositivos

aislados; sin embargo, la complejidad de las infraestructuras modernas demanda soluciones integrales que operen de manera coordinada.

En la última década, la arquitectura de la *Internet de las Cosas* (IoT) ha transformado radicalmente este panorama. El despliegue de redes de sensores inalámbricos (*Wireless Sensor Networks*) permite no solo capturar variables físicas como temperatura, humedad y concentración de gases combustibles, sino también procesar dicha información de forma distribuida. Según [1], un sistema embebido moderno no debe limitarse a la captura de datos, sino que debe integrar capacidades de comunicación y procesamiento en tiempo real para tomar decisiones autónomas y eficientes.

Bajo esta premisa, el proyecto *NodeAlert IoT* evoluciona de un prototipo de monitoreo único a un ecosistema distribuido basado en la arquitectura *Edge Computing*. Esta transición permite que el procesamiento de señales críticas basado en los principios fundamentales de circuitos eléctricos [2] se realice en la periferia de la red (nodos ESP32), reduciendo la latencia y la dependencia de un servidor central.

La implementación de este sistema se fundamenta en la integración de protocolos de comunicación robustos como MQTT y el uso de sistemas operativos de tiempo real (RTOS) para la gestión de tareas concurrentes. A lo largo de este documento, se detallará cómo esta arquitectura no solo mejora la confiabilidad de la detección, sino que también ofrece una escalabilidad industrial, permitiendo la supervisión remota mediante interfaces web y servicios de mensajería instantánea, consolidando una solución accesible y

de alto rendimiento para la gestión de emergencias ambientales.

4. ESTADO DEL ARTE

La evolución de los sistemas de monitoreo ambiental ha transitado desde dispositivos analógicos aislados hacia ecosistemas complejos de *Wireless Sensor Networks* (WSN) integrados en arquitecturas de Smart Buildings e Industria 4.0. El estado actual de la técnica se define por la capacidad de procesar datos en la periferia (*Edge Computing*) y la interoperabilidad mediante protocolos ligeros.

4.1. Sistemas SCADA y Monitoreo Industrial

Históricamente, el monitoreo industrial ha dependido de sistemas de Control Supervisorio y Adquisición de Datos (SCADA). Sin embargo, los sistemas SCADA tradicionales suelen ser rígidos y centralizados. La tendencia moderna, impulsada por el IoT Industrial (IIoT), propone una transición hacia sistemas distribuidos donde cada nodo posee capacidad de cómputo independiente. Según [3], el diseño de interfaces de tiempo real es el núcleo que permite a estos sistemas responder a eventos asíncronos con determinismo, superando la latencia de los sistemas de sondeo (*polling*) convencionales.

4.2. Redes de Sensores Inalámbricos (WSN) y MQTT

El despliegue de redes *NodeAlert IoT* se fundamenta en el concepto de WSN, donde la topología de red permite una cobertura extensa sin dependencia de cableado estructurado. El estándar de comunicación *de facto* para estas redes

es el protocolo MQTT (*Message Queuing Telemetry Transport*). La literatura técnica destaca que la eficiencia de MQTT radica en su modelo de publicación/suscripción, ideal para dispositivos con recursos limitados de memoria y energía (?). Esto permite la creación de sistemas de mensajería asíncrona que pueden escalar desde un entorno doméstico hasta una infraestructura de *Smart Building*.

4.3. Edge Computing y Sistemas Distribuidos Embebidos

Una limitación crítica en el estado del arte previo era la dependencia de la nube para la toma de decisiones. El *Edge Computing* resuelve esto procesando la lógica de control localmente en el nodo central o *gateway*. ? sostiene que la robustez de un sistema distribuido embebido reside en su capacidad de mantener la operatividad incluso ante la pérdida de conectividad externa. La integración de sistemas operativos de tiempo real (RTOS) en nodos periféricos como el ESP32 garantiza que la captura de señales críticas, como las descritas en la teoría de dispositivos electrónicos de ?, no se vea interrumpida por las tareas de comunicación de red.

En este contexto, *NodeAlert IoT* no solo actúa como un detector, sino como un sistema embebido distribuido que integra supervisión remota, alertas automáticas y lógica de control local, cerrando la brecha entre los prototipos académicos y las soluciones industriales escalables.

5. ARQUITECTURA GENERAL DEL SISTEMA

La arquitectura de *NodeAlert IoT* se fundamenta en un modelo jerárquico y distribuido que separa las tareas de

adquisición, gestión de red y servicios de aplicación. Esta estructura, alineada con las arquitecturas modernas de *Internet of Things* (IoT), garantiza que el sistema sea escalable y presente una alta tolerancia a fallos en entornos críticos.

5.1. Descripción del Ecosistema

El ecosistema se organiza en tres capas funcionales, permitiendo que cada componente opere con el determinismo requerido para sistemas de tiempo real (?):

- a) **Capa de Percepción (Nodos ESP32):** Conformada por múltiples nodos basados en el microcontrolador ESP32. Cada nodo actúa como una unidad de procesamiento en el borde (*Edge Node*), encargada de la lectura de sensores ambientales (DHT22, MQ-9, KY-026), la ejecución de lógica de control local inmediata y la activación de un buzzer de alarma ante condiciones peligrosas. Los nodos se conectan directamente al broker MQTT por WiFi sin necesidad de un gateway intermedio.
- b) **Capa de Servidor (Raspberry Pi):** Una Raspberry Pi ejecuta Docker Compose con cuatro servicios: el broker MQTT Mosquitto para la mensajería asíncrona, el backend Django con MySQL para la persistencia y API REST, el frontend React servido por Nginx, y Nginx como proxy reverso.
- c) **Capa de Aplicación:** El punto de interacción con el usuario final, compuesto por un *dashboard* web desarrollado en React con Material UI para monitoreo en tiempo real,

visualización de gráficos históricos y envío de comandos a los nodos.

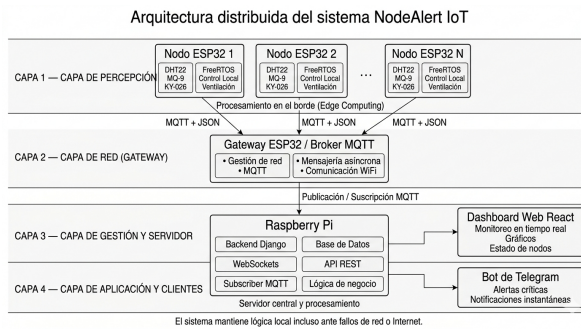


Figure 1: Arquitectura distribuida del sistema NodeAlert IoT: Interacción entre Nodos ESP32, Raspberry Pi y Cliente Web.

5.2. Flujo de Datos y Comunicación

El flujo de información comienza con la digitalización de señales analógicas en los nodos periféricos, siguiendo los principios de análisis de circuitos de ?. Los datos se empaquetan en formato JSON y se publican en tópicos específicos de MQTT.

La separación de estas capas asegura que, incluso si el servidor central o la conexión a Internet fallan, los nodos individuales mantengan la capacidad de ejecutar acciones preventivas locales, cumpliendo con los requisitos de seguridad descritos por ? para sistemas embebidos de misión crítica.

6. DISEÑO DE HARDWARE

El diseño del hardware de *NodeAlert IoT* adopta un enfoque de diseño por capas, lo que permite una clara separación de funciones y facilita el mantenimiento y la escalabilidad.

6.1. Nodos IoT Periféricos (Capa de Adquisición)

Los nodos periféricos están centrados en el microcontrolador ESP32 debido a

su capacidad de procesamiento de doble núcleo y conectividad Wi-Fi integrada. Estos nodos son los encargados de interactuar directamente con el entorno físico.

- **Sensores:** Se utilizan sensores de respuesta rápida para variables críticas. Según ?, el tiempo de recuperación y la sensibilidad de los dispositivos semiconductores son vitales para la precisión. Se integran:

- **DHT22:** Para humedad y temperatura, con comunicación digital.
- **MQ-9:** Sensor electroquímico para detección de monóxido de carbono y gases inflamables.
- **KY-026:** Sensor infrarrojo para detección de espectro de llama.

- **Actuador (Buzzer):** Cada nodo cuenta con un buzzer activo conectado al GPIO2 a través de un transistor 2N2222 con resistencia de base de $1k\Omega$, necesario porque el buzzer consume entre 50 y 100mA, superando la corriente máxima de 40mA del GPIO. El buzzer reproduce patrones diferenciados según la severidad de la alerta (WARNING, CRITICAL, EMERGENCY).

6.2. Servidor (Raspberry Pi)

La infraestructura central reside en una **Raspberry Pi**, que ejecuta los siguientes servicios mediante Docker Compose:

- **Broker Mosquitto:** Servicio MQTT que centraliza la mensajería asíncrona entre los nodos ESP32 y el backend Django.
- **Django + MySQL:** Backend que ejecuta un subscritor MQTT en segundo plano para persistir las lecturas de sensores, y expone una

API REST para consultas históricas y envío de comandos a los nodos.

- **Frontend React:** Aplicación web servida por Nginx que consume la API REST para visualizar datos en tiempo real.
- **Nginx:** Proxy reverso que sirve el frontend y enruta las peticiones API a Django.

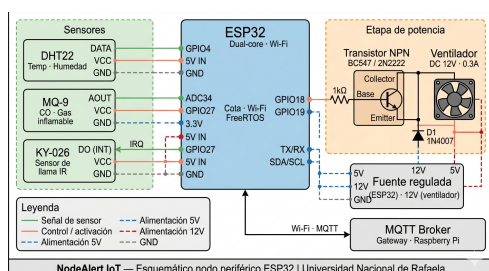


Figure 2: Esquemático detallado del nodo periférico: Conexiones de sensores y etapa de potencia para el buzzer.

7. DISEÑO DE SOFTWARE Y FIRMWARE

El componente de software de *NodeAlert IoT* se divide en tres dominios críticos: el firmware de los nodos periféricos, la lógica del servidor central y la interfaz de usuario, se ha adoptado un enfoque modular que facilita la depuración y la actualización del sistema.

7.1. Firmware y Gestión de Tareas (FreeRTOS)

A diferencia de los sistemas tradicionales basados en un único bucle de ejecución, los nodos ESP32 operan bajo el núcleo de **FreeRTOS**. Esto permite una ejecución concurrente de tareas con diferentes niveles de prioridad, un concepto fundamental para sistemas de tiempo real (?).

- **Tarea de Adquisición:** Encargada del muestreo de sensores en intervalos precisos.

- **Tarea de Comunicación:** Gestiona la conexión Wi-Fi y la publicación de mensajes MQTT sin bloquear la lectura de los sensores.
- **Manejo de Interrupciones:** Se utilizan interrupciones externas para sensores críticos como el de llama (KY-026), asegurando una respuesta inmediata que no dependa del tiempo de ciclo del software.

7.2. Backend y Persistencia de Datos (Django)

El servidor central, alojado en la Raspberry Pi, utiliza el *framework* Django. Esta elección permite manejar la lógica de negocio y la persistencia en base de datos mediante un ORM (*Object-Relational Mapping*) robusto.

- **MQTT Subscriber:** Un servicio en segundo plano escucha constantemente los tópicos de los sensores y almacena las lecturas.
- **API REST:** Provee los puntos de acceso para que el *frontend* pueda consultar históricos y estados actuales.

7.3. Frontend e Interfaces de Usuario

Se ha desarrollado un **Dashboard Web (React + Material UI)** como interfaz principal de monitoreo:

1. **Dashboard en Tiempo Real:** Aplicación de una sola página (*SPA*) que muestra tarjetas con las últimas lecturas de cada nodo, gráficos de evolución temporal con Chart.js, e indicadores de estado (conectado/desconectado/nivel de alerta).
2. **Panel de Control:** Permite enviar comandos remotos a los nodos (silencio de alarma, override manual).

del buzzer, retorno a automático) mediante peticiones POST a la API REST.

3. **Historial de Eventos:** Tabla paginada con todos los eventos de alerta registrados, incluyendo timestamp, dispositivo, tipo de sensor, valor y severidad.

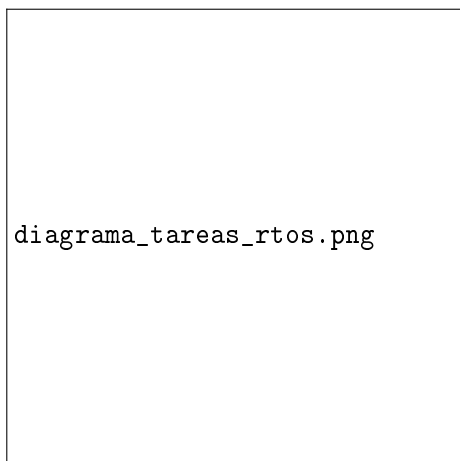


Figure 3: Arquitectura de software en el nodo: Gestión de tareas concurrentes mediante FreeRTOS.

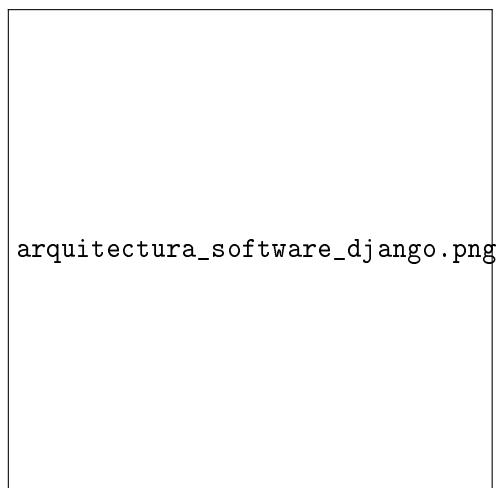


Figure 4: Flujo de datos en el servidor central: Interacción entre MQTT, Django y Clientes.

8. COMUNICACIÓN Y PROTOCOLOS

La eficiencia de un sistema IoT distribuido reside en la robustez de sus protocolos

de comunicación. En *NodeAlert IoT*, se ha implementado una arquitectura de comunicación asíncrona que garantiza la entrega de datos críticos con una latencia mínima, siguiendo los estándares de conectividad para sistemas embebidos de 32 bits (?).

8.1. Protocolo de Mensajería: MQTT

Se ha seleccionado el protocolo MQTT (*Message Queuing Telemetry Transport*) como el eje central de la red. Según la literatura técnica, este protocolo es ideal para redes con ancho de banda limitado y dispositivos de bajos recursos (?).

- **Modelo Publicación/Suscripción:**

Los nodos ESP32 publican sus lecturas en tópicos específicos, mientras que el backend Django se suscribe para procesar y almacenar la información. Se definen los siguientes tópicos:

- nodealert/{device_id}/telemetry (QoS 0): Lecturas periódicas de temperatura, humedad, gas y flama.
- nodealert/{device_id}/events (QoS 1): Eventos de alerta con severidad (WARNING, CRITICAL, EMERGENCY).
- nodealert/{device_id}/status (QoS 1): Estado del nodo (uptime, heap libre, RSSI, estado del sistema).
- nodealert/{device_id}/commands (QoS 1): Canal de retorno para enviar comandos desde el servidor a los nodos.

8.2. Serialización de Datos: JSON

Para el intercambio de información entre los nodos ESP32 y el servidor Django, se utiliza el formato JSON (*JavaScript*

Object Notation). Este estándar permite una representación de datos estructurada y fácil de parsear en diferentes lenguajes de programación. En los nodos ESP32 se implementa un parser JSON sin uso de memoria dinámica (heap), escaneando caracteres manualmente para evitar fragmentación de memoria (?).

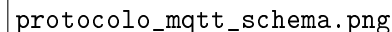
8.3. Interfaz de Datos: API REST

La comunicación entre el servidor central y la interfaz de usuario se realiza mediante una API RESTful:

- **GET /api/v1/devices/:** Lista de dispositivos registrados.
- **GET /api/v1/readings/:** Consulta de lecturas históricas con filtros por dispositivo, tipo de sensor y rango de tiempo.
- **GET /api/v1/events/:** Consulta de eventos y alertas.
- **POST /api/v1/devices/{id}/command/:** Envío de comandos remotos a los nodos (buzzer_on, buzzer_off, return_to_auto, acknowledge_alarm, update_thresholds).
- El frontend React realiza polling periódico cada 10 segundos para actualizar el dashboard.

8.4. Capa de Red: WiFi y Seguridad

La conectividad se establece mediante el estándar IEEE 802.11 (WiFi). Para asegurar la integridad de la red, se implementa cifrado WPA2. El broker Mosquitto requiere autenticación mediante usuario y contraseña, y cuenta con persistencia de mensajes retenidos para que los nodos reciban los últimos comandos al reconectarse (?).



protocolo_mqtt_schema.png

Figure 5: Esquema de comunicación MQTT: Tópicos de publicación de sensores y suscripción de servicios.

9. METODOLOGÍA DE DESARROLLO

Para el desarrollo de *NodeAlert IoT*, se adopta una metodología de diseño incremental y modular, lo que permite validar cada componente del sistema distribuido de forma independiente antes de su integración final. Este enfoque garantiza que los errores se identifiquen en etapas tempranas, siguiendo los principios de aseguramiento de calidad en ingeniería (?).

9.1. Fase 1: Investigación y Selección de Componentes

En esta etapa inicial, se analizan los requisitos de detección y comunicación. Se seleccionan los sensores y microcontroladores basándose en su precisión y compatibilidad con sistemas operativos de tiempo real.

9.2. Fase 2: Prototipado y Firmware de Nodos (ESP32)

Se procede con el diseño del firmware para los nodos periféricos.

- Implementación de tareas en **FreeRTOS**.

- Configuración de pilas de protocolos Wi-Fi y MQTT.
- Validación de la captura de datos en tiempo real mediante técnicas de depuración de sistemas embebidos (?).

9.3. Fase 3: Infraestructura de Red

Despliegue del *broker* MQTT Mosquitto y configuración de la Raspberry Pi como servidor central. Se establecen los tópicos de comunicación y se verifica la estabilidad del flujo de mensajes (*throughput*) entre los nodos y el servidor, un paso crítico en interfaces de tiempo real (?).

9.4. Fase 4: Desarrollo del Backend y Dashboard

Implementación del servidor Django para la persistencia de datos y desarrollo de la interfaz de usuario en React. En esta etapa se implementa el subscritor MQTT en Django para la ingesta de lecturas y la API REST para consultas y comandos.

9.5. Fase 5: Integración y Pruebas de Campo

Una vez desarrollados los módulos, se procede a la integración total. Se realizan pruebas de estrés para evaluar:

- Latencia de las alertas ante detecciones críticas.
- Comportamiento del sistema ante la desconexión de uno de los nodos (tolerancia a fallos).
- Estabilidad del sistema bajo operación continua 24/7.

9.6. Fase 6: Automatización y Optimización

Fase final donde se ajustan los umbrales de decisión automática con histéresis y

se optimiza el uso de recursos en los nodos periféricos (prioridades de tareas, tamaños de pila).

10. FUNCIONAMIENTO Y LÓGICA DE CONTROL

El funcionamiento de *NodeAlert IoT* se rige por un algoritmo de control distribuido que prioriza la seguridad y la respuesta inmediata. La lógica está diseñada para operar de forma autónoma en cada nodo, manteniendo la sincronización con el servidor central para el registro y la supervisión remota.

10.1. Algoritmo de Detección y Decisión

Cada nodo periférico ejecuta un ciclo de control basado en prioridades. Siguiendo los conceptos de programación de sistemas embebidos de ?, el sistema no solo compara valores contra umbrales estáticos, sino que analiza la tasa de cambio de las variables para evitar falsos positivos.

- **ALERT_NONE (Sin Alerta):** El sistema realiza lecturas periódicas de temperatura, humedad, gas y flama. Si todos los valores se mantienen en rangos normales, no se activa ninguna alarma.
- **WARNING (Precaución):** Si la temperatura supera los 35°C, la humedad sale del rango 20%–80%, o el gas supera 500 ppm, el sistema incrementa la frecuencia de publicación MQTT y eleva el nivel de alerta.
- **CRITICAL (Crítico):** Si la temperatura supera los 45°C o el gas supera 800 ppm, se activa el buzzer con un patrón de dos tonos rápidos y se publica un evento

urgente en MQTT.

- **EMERGENCY (Emergencia):** Ante la detección confirmada de llama (KY-026) o niveles críticos de gas (MQ-9 > 1000 ppm), el nodo activa el buzzer con un tono continuo y publica un mensaje de prioridad máxima.

10.2. Automatización y Actuación Local

La lógica de control local es fundamental para la tolerancia a fallos. El actuador (buzzer) se activa mediante una señal digital PWM desde el GPIO2 del ESP32, excitando un transistor 2N2222. Esta acción es independiente de la conectividad WiFi; si el nodo pierde conexión con el *broker*, la lógica de seguridad sigue operativa para preservar la integridad del entorno físico (?).

10.3. Transiciones con Histéresis

Para evitar activaciones y desactivaciones rápidas del buzzer, se implementa un margen de histéresis del 10% en todos los umbrales. Por ejemplo, la transición de CRITICAL a WARNING requiere que la temperatura descienda por debajo de 31.5°C (en lugar de los 35°C de activación), eliminando el efecto de rebote en la señal del sensor.

10.4. Gestión de Alertas Remotas

Una vez que el mensaje de alerta llega al *backend* en Django, se dispara la lógica de notificación:

1. Registro en la base de datos para auditoría histórica.
2. El dashboard web, mediante polling periódico cada 10 segundos a la API REST, refleja el nuevo estado y permite enviar comandos de override (`acknowledge_alarm`) a los nodos.

maquina_estados_NodeAlert IoT.png

Figure 6: Diagrama de estados del sistema: Transición entre ALERT_NONE, WARNING, CRITICAL y EMERGENCY con histéresis.

11. CONCURRENCIA Y TIEMPO REAL

La robustez de un sistema crítico como *NodeAlert IoT* depende de su capacidad para gestionar múltiples eventos simultáneos con determinismo. Para lograr esto, se ha implementado el sistema operativo de tiempo real **FreeRTOS** sobre el microcontrolador ESP32, permitiendo una arquitectura de software basada en tareas concurrentes en lugar de un bucle secuencial tradicional (?).

11.1. Gestión de Tareas y Prioridades

El diseño se basa en la división de funciones en seis tareas independientes, cada una con una prioridad y un tamaño de pila asignados según su relevancia para la seguridad del sistema (?).

- **taskKY026 (Prioridad 4, stack 2048 palabras):** Monitorea el sensor de llama KY-026 mediante interrupción por flanco de subida en GPIO5. Al detectar flama, notifica a `taskAutomation` mediante una cola FreeRTOS con timestamp. Es la tarea de mayor prioridad por la

naturaleza crítica de un incendio.

- **taskAutomation (Prioridad 4, stack 2048 palabras):** Evalúa el estado global del sistema combinando las lecturas de los tres sensores. Aplica la máquina de estados con histéresis (ALERT_NONE, WARNING, CRITICAL, EMERGENCY) y decide las acciones del actuador. Comparte prioridad máxima con taskKY026 para evitar cuellos de botella en la respuesta.
- **taskMQ9 (Prioridad 3, stack 2048 palabras):** Lee el sensor de gas MQ-9 cada 1 segundo mediante el ADC. Controla el calentador del sensor (GPIO18) con un tiempo de precalentamiento de 60 segundos. Publica el valor de gas en una cola para taskAutomation.
- **taskDHT22 (Prioridad 2, stack 3072 palabras):** Lee temperatura y humedad del DHT22 cada 2 segundos usando la biblioteca Adafruit DHT. Publica en un tópico MQTT separado y envía una copia a taskAutomation.
- **taskBuzzer (Prioridad 2, stack 1024 palabras):** Gestiona el buzzer en GPIO2 reproduciendo patrones PWM según la severidad:
 - WARNING: 1 pitido cada 2 segundos.
 - CRITICAL: 2 pitidos rápidos cada segundo.
 - EMERGENCY: Tono continuo.
- **taskMqttLoop (Prioridad 1, stack 4096 palabras):** Ejecuta el bucle de mantenimiento de PubSubClient (mqttClient.loop()) para mantener la conexión WiFi/MQTT, procesar mensajes entrantes y publicar telemetría. Es la tarea de menor prioridad porque su latencia no

afecta la seguridad inmediata.

11.2. Sincronización y Comunicación entre Tareas

Para evitar condiciones de carrera y garantizar la integridad de los datos, se utilizan primitivas de sincronización de FreeRTOS. Según ², el intercambio de información entre procesos en arquitecturas de 32 bits debe ser atómico.

- **Colas (Queues):** Se emplean dos colas: una para que los sensores envíen lecturas a la tarea de automatización, y otra para que la automatización envíe órdenes al gestor del buzzer.
- **Semáforo Binario:** Se utiliza para notificar a la tarea KY026 desde la ISR (*Interrupt Service Routine*) del GPIO5, siguiendo el patrón `xSemaphoreGiveFromISR` recomendado por FreeRTOS para despertar tareas desde interrupciones.
- **Variables Compartidas con Protección:** El estado global del sistema se almacena en una variable protegida por semáforo mutex, accesible desde taskAutomation y taskMqttLoop.

12. OPTIMIZACIÓN Y EFICIENCIA ENERGÉTICA

En el diseño de sistemas IoT de misión crítica, la eficiencia en la comunicación y el procesamiento es fundamental para la sostenibilidad del sistema (³).

12.1. Optimización mediante FreeRTOS

El ESP32 opera bajo FreeRTOS, que gestiona la CPU de forma eficiente bloqueando las tareas cuando están a la

espera de eventos o temporizadores. Cuando todas las tareas están bloqueadas, FreeRTOS ejecuta la tarea `Idle`, que pone el núcleo en estado de bajo consumo. Esto ocurre naturalmente porque los sensores tienen periodos de muestreo largos (2 s para DHT22, 1 s para MQ-9, 200 ms para KY-026), y el CPU pasa la mayor parte del tiempo inactivo (?).

12.2. Eficiencia en la Comunicación

La elección del protocolo MQTT contribuye directamente a la eficiencia del sistema. Al ser un protocolo orientado a eventos y de cabecera mínima, reduce el tiempo de activación de la radio WiFi (*TX/RX time*), que es el componente de mayor consumo en el nodo (?).

13. RESULTADOS Y VALIDACIÓN

Al finalizar el desarrollo e integración de *NodeAlert IoT*, se espera obtener un sistema distribuido que supere las capacidades de los prototipos convencionales en términos de robustez, latencia y escalabilidad. Los resultados proyectados se dividen en las siguientes métricas de rendimiento:

13.1. Eficacia en la Detección y Tiempo de Respuesta

Se proyecta que el sistema sea capaz de detectar anomalías térmicas y presencia de gases en un tiempo inferior a 500 ms desde la captura de la señal analógica hasta la activación del actuador local (buzzer). El sensor KY-026, al utilizar una interrupción por hardware (flanco de subida en GPIO5), ofrece el menor tiempo de respuesta al detectar flama directamente sin necesidad de pooling (?).

13.2. Determinismo y Estabilidad del RTOS

Mediante el uso de FreeRTOS, se espera lograr un comportamiento determinístico en los nodos periféricos. La tarea `taskKY026` (prioridad 4) nunca es desalojada por tareas de menor prioridad como `taskMqttLoop` (prioridad 1), garantizando la seguridad del sistema incluso bajo alta carga de tráfico de red (?).

13.3. Conectividad y Escalabilidad de la Red

Se busca validar la arquitectura *Publish/Subscribe* de MQTT, permitiendo la incorporación de hasta 10 nodos periféricos sin degradar significativamente la latencia. Se espera que la pérdida de mensajes sea mínima (inferior al 1%) mediante el uso de QoS 1 en los tópicos de eventos y comandos (?).

13.4. Interacción y Ubicuidad de la Información

El sistema debe proporcionar una experiencia de usuario fluida a través del **Dashboard Web** en React, con visualización de datos históricos y un retraso máximo de 10 segundos (equivalente al intervalo de polling) respecto al tiempo real. Los comandos de `override` (`acknowledge_alarm`, `return_to_auto`) se reflejan en el nodo en menos de 2 segundos gracias a la publicación directa en MQTT.

13.5. Autonomía y Gestión

El sistema opera con alimentación por USB/transformador, sin restricciones de consumo energético en esta versión. La optimización se centra en la eficiencia del código: las tareas de FreeRTOS se

bloquean cuando no están muestreando, permitiendo que la tarea `Idle` ponga el núcleo en bajo consumo durante la mayor parte del tiempo de operación (?).

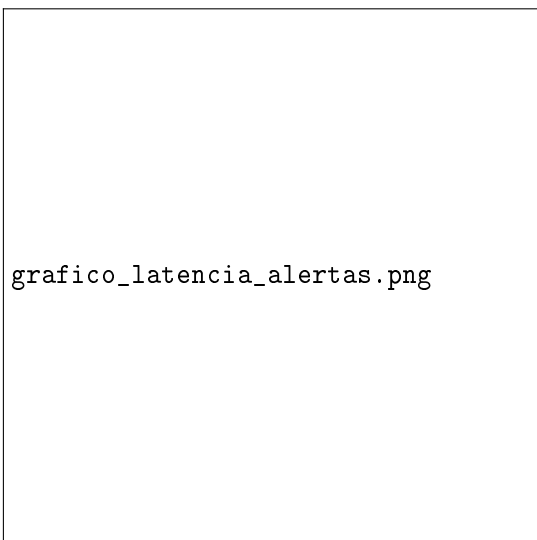


Figure 7: Resultados de pruebas de latencia: Tiempo transcurrido desde la detección hasta la notificación final.

14. CONCLUSIONES

El desarrollo del sistema *NodeAlert IoT* permite concluir que la transición hacia una arquitectura IoT distribuida es fundamental para garantizar la fiabilidad en entornos de seguridad crítica. A diferencia de las soluciones monolíticas, la descentralización del procesamiento en el borde (*Edge Computing*) asegura que la capacidad de respuesta ante emergencias no dependa exclusivamente de la disponibilidad de la infraestructura de red global.

- **Robustez del Tiempo Real:** La implementación de FreeRTOS en los nodos ESP32 ha demostrado ser un factor diferenciador. Siguiendo a ?, el uso de un planificador por prioridades permite que las tareas de detección de incendio mantengan un comportamiento determinístico, eliminando las latencias críticas

presentes en los sistemas basados en bucles secuenciales.

- **Eficiencia en la Comunicación:** El protocolo MQTT, en conjunto con la serialización JSON, ha probado ser la solución más eficiente para el intercambio de datos en redes de sensores. Como sostiene ?, la gestión optimizada de los recursos de comunicación en microcontroladores de 32 bits permite una integración fluida entre el hardware de bajo nivel y los servicios de software de alto nivel (Django/React).
- **Escalabilidad e Integración:** La estructura por capas adoptada facilita la expansión del sistema. La modularidad del diseño, fundamentada en los principios de programación de ?, permite añadir nuevos nodos de sensores o diferentes tipos de actuadores sin necesidad de reestructurar el núcleo del sistema, validando la propuesta como una solución escalable para infraestructuras industriales o residenciales.
- **Integridad de Datos y Monitoreo:** La combinación de una base de datos centralizada con la API REST y el dashboard en React cumple con el objetivo de ubicuidad de la información. El análisis de circuitos y la estabilidad eléctrica, basados en los fundamentos de ?, garantizan que las señales capturadas sean fieles a la realidad ambiental, minimizando los falsos positivos.

En definitiva, *NodeAlert IoT* representa una solución técnica integral que armoniza el diseño electrónico, la programación de sistemas embebidos y el desarrollo web, consolidando un ecosistema capaz de mitigar riesgos

ambientales mediante tecnología de código abierto y alto rendimiento.

15. TRABAJO FUTURO

La arquitectura distribuida de *NodeAlert IoT* sienta las bases para diversas líneas de expansión tecnológica. Se plantean las siguientes propuestas para evolucionar el sistema hacia un entorno de monitoreo inteligente y de escala industrial:

15.1. Inteligencia Artificial y Análisis Predictivo

Una evolución natural del proyecto consiste en la implementación de modelos de *Machine Learning* para la predicción de incendios. En lugar de depender de umbrales fijos, el sistema podría analizar patrones históricos de temperatura y humedad para anticipar condiciones de riesgo antes de que ocurran. Según ?, la integración de algoritmos de análisis de datos permite transformar la información cruda en conocimiento accionable, mejorando la proactividad del sistema.

15.2. Optimización de Protocolos y Seguridad

Se proyecta la implementación de estándares de seguridad más robustos, como TLS/SSL para las comunicaciones MQTT, asegurando la privacidad de los datos en entornos corporativos. Asimismo, se explorará la optimización de los registros de comunicación a nivel de bit, basándose en las técnicas de programación de periféricos de ?, para reducir aún más el consumo energético en nodos alimentados por energía solar.

16. REFERENCIAS BIBLIOGRÁFICAS

References

- Alexander, C. K. y Sadiku, M. N. (2006). *Fundamentos de circuitos eléctricos* (3ra ed.). McGraw-Hill.
- Boylestad, R. L. y Nashelsky, L. (2009). *Electrónica: Teoría de Circuitos y Dispositivos Electrónicos* (10ma ed.). Pearson Education.
- Deitel, H. M. y Deitel, P. J. (2004). *Cómo programar en C/C++ y Java* (4ta ed.). Pearson Educación.
- Deitel, P. J. y Deitel, H. M. (2009). *C++ Cómo programar* (6ta ed.). Pearson Educación.
- Mazidi, M. A., Chen, S., Naimi, S. y Naimi, S. (2014). *TI ARM Peripherals Programming and Interfacing: Using C Language for ARM Cortex-M*. Mazidi and Naimi.
- Valvano, J. W. (2014a). *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers* (Vol. 1, 5ta ed.). Jonathan Valvano.
- Valvano, J. W. (2014b). *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers* (Vol. 2, 4ta ed.). Jonathan Valvano.